From C to Silicon

EE5203 companion — the toolchain, memory, and boot process of an STM32 program

Basri Erdogan

Table of contents

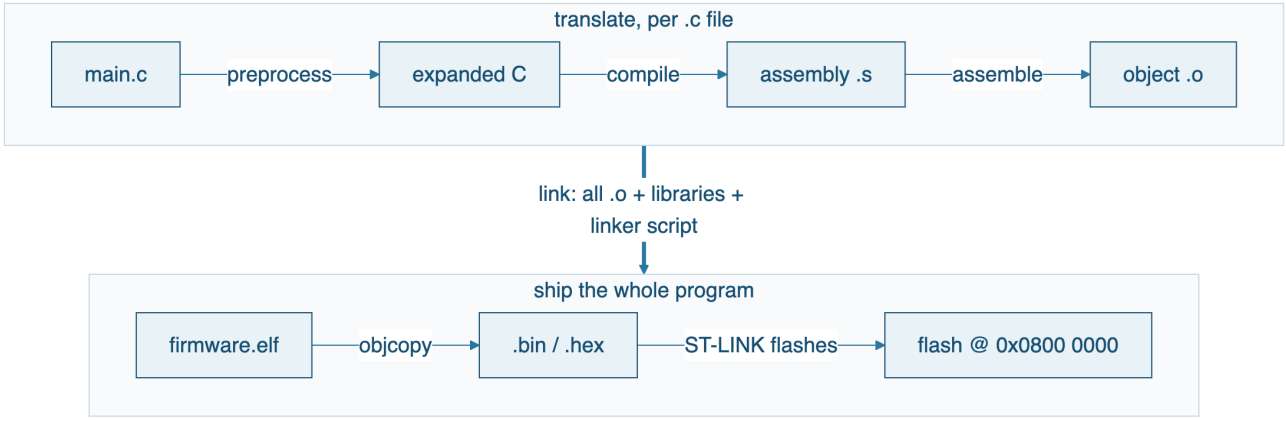
A companion reference for EE5203 Microprocessors. It answers the questions the lectures deliberately defer: what actually happens between saving `main.c` and the LED blinking. Read it alongside L01 (the toolchain), L04 (registers and memory-mapped I/O), L06 (the vector table), and the optional assembly deck (the bridge to assembly — available in CATS). You need to know C; you do not need to know anything about compilers, linkers, or boot ROMs — that is what this document is for.

Every concrete number in this companion is real: the memory sizes and addresses come from the STM32F401RE on your Nucleo board, the linker-script and startup-code excerpts come from ST's actual generated files, and the tool names are the ones in the course toolchain (STM32CubeMX, CMake + Ninja, `arm-none-eabi-gcc` 14.3, and the ST-LINK tools from STM32CubeCLT, driven from VS Code).

1 The big picture

1.1 One sentence, six machines

When you press **Build**, your C source passes through a pipeline of programs, each of which consumes one representation of your program and produces a lower-level one:



Reading right to left is just as instructive: the chip can only execute machine code stored at known addresses (Section 2), so *something* must decide those addresses (the linker, Section 3), *something* must produce position-correct machine code (the compiler and assembler, Section 3), and *something* must physically write the result into flash (the programmer, Section 6). The pipeline exists because each of those jobs is genuinely different work.

> **!** Why this matters
>
> Every mysterious build error lives at a specific stage of this pipeline, and the stage tells you where to look. `undefined reference to 'foo'` is a **linker** error — the compiler was perfectly happy, so no amount of staring at `#include` lines will fix it. `unknown type name 'uint8_t'` is a **compiler** error — the linker never even ran. Learning which machine spoke saves hours.

1.2 Who does what

Stage	Program (in our toolchain)	Consumes	Produces	Typical failure
Preprocess	`arm-none-eabi-gcc -E` (built in)	`.c` + headers	expanded C	missing header
Compile	`arm-none-eabi-gcc`	one `.c`	one `.o`	syntax/type errors

Stage	Program (in our toolchain)	Consumes	Produces	Typical failure
Assemble	`arm-none-eabi-as` (invoked by gcc)	`.s`	`.o`	rarely fails
Link	`arm-none-eabi-ld` (via gcc)	all `.o` + libraries + linker script	`.elf`	undefined/duplicate symbols, region overflow
Convert	`arm-none-eabi-objcopy`	`.elf`	`.bin`, `.hex`	—
Flash	STM32CubeProgrammer / GDB server over ST-LINK	`.elf`/`.bin`	programmed flash	wrong address, locked flash

> ⚠ Common misconception
>
> "The compiler builds my project." It does not. The compiler sees **one `.c` file at a time** and has no idea any other file exists. It is the *linker* that first sees your whole program — which is why a function can compile perfectly and still not exist at link time, and why `static` (invisible outside this file) is enforced at the link stage, not by a compiler pass over your whole project.

The build *orchestrator* — the program that decides which files need recompiling and calls the pipeline in order — is a separate tool again. In this course that is **CMake** (which generates the build plan) and **Ninja** (which executes it), both bundled in STM32CubeCLT and driven by the VS Code build task. CubeMX writes the initial `CMakeLists.txt` when it generates your project. The full keypress-to-LED story is traced in Section 7.

2 The memory your program lives in

2.1 One address space, several very different memories

A Cortex-M4 has a single, flat 32-bit address space: every byte of flash, every byte of RAM, and — as you have used since L04 — every peripheral register has an address in the same 4 GB map. On the STM32F401RE the regions that matter to us are:

Region	Start address	Size	What it is
Flash	`0x0800 0000`	512 KB	non-volatile; your code and constants; survives power-off
SRAM	`0x2000 0000`	96 KB	volatile; variables, stack, heap; blank at power-up
Peripherals	`0x4000 0000`	—	GPIO, TIM, ADC… — the SFRs of L04–L10
System memory	`0x1FFF 0000`	30 KB	ST's **boot ROM**: the built-in bootloader (Section 6)
Cortex-M core	`0xE000 0000`	—	NVIC, SysTick, SCB — the L06 machinery

The authoritative map is one figure in the reference manual: RM0368 Rev 6, §2.3 *Memory map* — p. 38; flash organization is §3.3 — p. 45.

> **!** Why this matters
>
> Every linker decision (Section 3), the entire startup dance (Section 5), and the whole boot process (Section 6) are consequences of one asymmetry in this table: **flash keeps its contents without power but is slow and awkward to write; SRAM is fast and freely writable but wakes up containing garbage.** Your program must therefore *live* in flash and *work* in SRAM — and something has to bridge the two every single reset.

2.2 Where the pieces of a C program go

Think of your program as four kinds of bytes, each with a different relationship to that asymmetry:

- **Code** (`.text`) — instructions. Executed directly from flash; never changes at run time.

- **Constants** (`.rodata`) — string literals, `const` tables. Read directly from flash.
- **Initialized globals** (`.data`) — e.g. `uint32_t speed = 250;`. Must be *writable* (so they must live in SRAM) but must also *start with the right value after power-off* (so their initial values must be stored in flash). They get **two homes**, and the startup code copies one to the other (Section 5).
- **Zero-initialized globals** (`.bss`) — e.g. `uint32_t count;`. Cheaper: no flash copy needed, the startup code just fills the region with zeros.

The stack (function locals, return addresses — watched live in L11) and the heap (if you ever use `malloc`, which this course does not) also live in SRAM, at addresses chosen by the linker script (Section 3).

> ⚠ Common misconception
>
> "My variables are stored in flash, since that's where my program is." Only their *initial values* are. The variables themselves — the bytes your code reads and writes — are in SRAM. This is also why "512 KB of flash" does not mean you have 512 KB of workspace: your working memory is the 96 KB of SRAM, and big arrays exhaust it long before flash fills up.

> ℹ Course tie-in
>
> This is L04's punchline extended: `GPIOA->ODR` was never special syntax — it is a pointer to address `0x4002 0014`, and the pin changed because a `str` instruction (the assembly deck's subject) wrote to that address. Peripherals are just the third kind of memory: not storage at all, but hardware listening on the bus.

3 From C to ELF: compiling and linking

3.1 Compilation: one file at a time

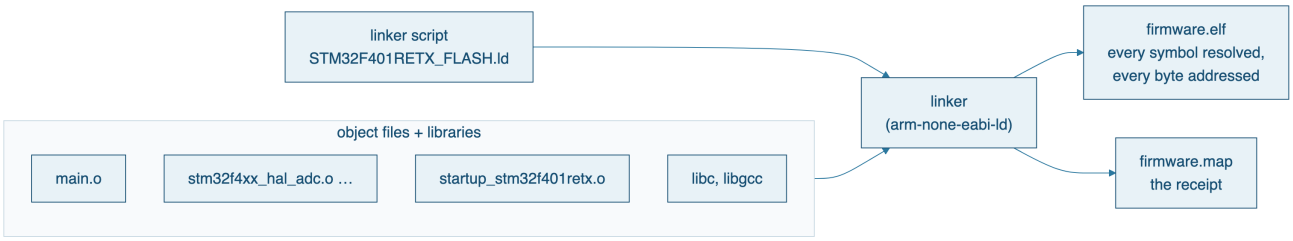
The compiler translates each `.c` file *independently* into an object file (`.o`). An object file already contains real Thumb-2 machine code (the same instructions you read in L11), but with two kinds of holes:

1. **Sections, not addresses.** The code is filed under named boxes — `.text`, `.rodata`, `.data`, `.bss` — with no decision yet about where in the 4 GB map (Section 2) each box will land.
2. **Symbols, not targets.** A call to `HAL_ADC_Start` compiles to a `bl` instruction with a blank destination and a note attached: *"patch this to wherever `HAL_ADC_Start` ends up."* That note is a **relocation**, and the name it mentions is a **symbol**.

You can see both with the tools already on your machine (`arm-none-eabi-objdump -d file.o` and `arm-none-eabi-nm file.o` — the U lines are symbols this file *uses* but does not *define*).

3.2 Linking: the whole program at last

The linker takes every `.o` plus the libraries (the HAL you compiled, the C runtime), and does exactly two jobs:



Job 1 — resolve: match every use of a symbol to its one definition. Zero matches → `undefined reference`. Two matches → `multiple definition`.

Job 2 — place: assign every section a real address. The rules for this come not from the C language but from a file you have so far ignored: the **linker script**. Here is the heart of the real one CubeMX generates for your board:

```
MEMORY
{
  RAM   (xrw) : ORIGIN = 0x20000000, LENGTH = 96K
  FLASH (rx)  : ORIGIN = 0x8000000,  LENGTH = 512K
}
```

Those two lines *are* Section 2's table, written in linker language. The `SECTIONS` part that follows then says, in effect: vector table first in FLASH, then all `.text`, then all `.rodata`; `.data` and `.bss` in RAM; `_estack` (the initial stack pointer) at the top of RAM. When you overflow a region you get the honest error `section '.bss' will not fit in region 'RAM'` — the linker script is where that judgment comes from.

> ⚠ Common misconception
>
> "`.data` is placed in RAM, so its initial values are in RAM." The linker gives `.data` **two addresses**: a *load* address in flash (where the initial values are stored, so they survive power-off) and a *run* address in RAM (where your code accesses the variables). In linker jargon these are the LMA and the VMA. Nothing moves the bytes between the two by itself — that is a real job, done by real instructions, every reset (Section 5). The symbol `_sidata` in the linker script marks the flash copy.

3.3 The ELF file and its flat cousins

The linker's output, `firmware.elf`, is a *container*: the placed bytes of every section **plus** everything a debugger needs — symbol names, source line numbers, section addresses. That is why the debugger can show you C source while the chip executes machine code, and why the `.elf` is what VS Code's launch flashes and debugs.

`objcopy` strips the container away, producing either a `.bin` (the raw bytes exactly as they will sit in flash, starting at `0x0800 0000`) or a `.hex` (the same bytes, with addresses spelled out in text). The chip never sees ELF — flash contains only the payload.

> ❗ Why this matters
>
> The `.map` file the linker writes next to the `.elf` is the single best build artifact you are not reading. It answers: *why is my image so big* (every object's contribution, byte by byte), *where did my variable go* (every symbol's address — the same addresses you see in the L11 Disassembly view), and *how close am I to the 96 KB SRAM ceiling*.

4 The build orchestra: CMake and Ninja

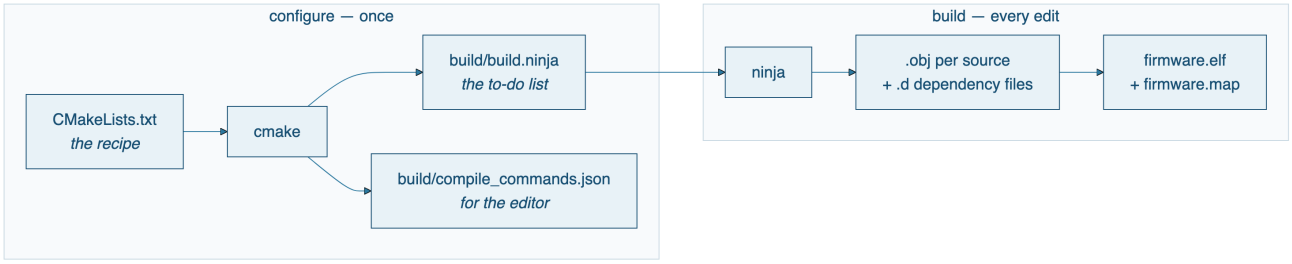
4.1 A planner and an executor

Section 3 described *what* the compiler and linker do to your code. This chapter answers a different question: **who tells them to do it, in what order, and why is the second build so much faster than the first?**

Two programs share that job, and neither of them compiles anything:

- **CMake is a planner.** It reads `CMakeLists.txt` — a *description* of the program: which source files, which include folders, which defines (`STM32F401xE`), which CPU flags (`-mcpu=cortex-m4`), which linker script. The description says **what** belongs to the program, never in which order to build it.
- **Ninja is an executor.** It reads the concrete to-do list CMake wrote and runs it: one compiler command per source file, then the link — and on every later run, **only** the commands whose inputs changed.

The kitchen version: *`CMakeLists.txt` is the recipe, CMake is the chef who writes the prep list, Ninja is the cook who only redoes the steps whose ingredients changed.*



4.2 What is on disk when: three snapshots

After CubeMX generates (before any build) the project folder is pure source — nothing machine-built yet:

```
project.ioc              CubeMX's own configuration – pins, clocks
Core/Inc, Core/Src       generated C: main.c, gpio.c, HAL config, IT file
startup_stm32f401retx.s  pre-main assembly – the startup code
STM32F401RETX_FLASH.ld   the linker script
CMakeLists.txt           the build recipe (+ cmake/ helpers, presets)
```

After the configure step (`cmake -B build -G Ninja`, run for you the first time the project opens) a disposable `build/` folder appears:

File	Role
`build.ninja`	the exhaustive to-do list: one build rule per source file
`CMakeCache.txt`	CMake's memory — compiler paths and options, so reconfiguring is instant
`compile_commands.json`	the exact compile command per file — **this is what feeds IntelliSense**; without it the editor red-underlines `#include "main.h"` while the real build succeeds

Still no firmware: configure plans, it does not cook.

After the build step (`ninja` — what ⇧⌘B and F5 actually run):

```
build/CMakeFiles/firmware.elf.dir/
    Core/Src/main.c.obj, gpio.c.obj, ..., startup_stm32f401retx.s.obj
    (+ one .d file each: "main.c.obj depends on main.h, stm32f4xx_hal.h, ...")
build/firmware.elf       the linked program - what F5 flashes (@sec-boot)
build/firmware.map       the linker's receipt (@sec-elf)
```

4.3 Why the second build takes milliseconds

Ninja records, next to every object file, a `.d` file listing **every header** that object's source included. On the next run it compares timestamps: edit `main.c` and exactly one compile command reruns, then the link. Edit a header and only the objects whose `.d` files name it rerun. Edit nothing and Ninja answers `no work to do` in a few milliseconds — that is the whole reason the edit-build-flash loop in the lab feels instant after the first build.

The link, by contrast, **always** reruns when any object changed: one new object can move every symbol's final address (Section 3).

> ⚠ Common misconception
>
> "CMake compiled my program." It never does — check `build.ninja` and you will find the `arm-none-eabi-gcc` command lines written out in full; CMake wrote them once at configure time, and Ninja replays them. This split also explains the two distinct failure smells: *configure errors* mention CMakeLists lines and missing tools; *build errors* mention your C code.

> ❗ Why this matters
>
> `build/` is entirely disposable — every byte in it can be regenerated from the sources. That gives you a clean, meaningful repair action the old IDE never made explicit: **delete `build/`, reconfigure, rebuild** cures any corrupted-build-state mystery. It is also why the lab's save-to-network script skips `build/`: you are saving the recipe, not the cooked meal.

5 Before main: the startup code

5.1 The contract C silently assumes

Every C program begins with promises already kept: initialized globals have their values, zeroed globals are zero, the stack works. On your laptop an operating system keeps those promises. On the F401RE **there is no operating system** — the promises are kept by about forty assembly instructions that CubeMX generated into your project and you have probably never opened: `startup_stm32f401retx.s`.

5.2 What the hardware does first

At reset the Cortex-M4 does exactly two fetches from the base of the vector table (L06's table — this is its origin story):

```
g_pfnVectors:
  .word  _estack          @ word 0: initial stack pointer
  .word  Reset_Handler    @ word 1: where to start running
  .word  NMI_Handler      @ word 2 onward: the L06 exception vectors
  ...
```

Word 0 $\rightarrow$ SP, word 1 $\rightarrow$ PC. That is the entire hardware boot protocol: *load the stack pointer, jump to the reset handler.* Everything after that is software.

5.3 Reset_Handler, line by line

This is the real generated code, lightly condensed — with the optional assembly deck behind you, you can now read it:

```
Reset_Handler:
  ldr  sp, =_estack        @ stack at the top of RAM (linker symbol)
  bl   SystemInit          @ minimal clock/system setup (C function)

  @ copy .data: flash (_sidata) -> RAM (_sdata.._edata)
  ldr  r0, =_sdata
  ldr  r1, =_edata
  ldr  r2, =_sidata
  ...                      @ a word-copy loop: ldr / str / adds / cmp / bcc
```

```
@ zero .bss (_sbss.._ebss)
...                     @ a store-zero loop

bl   __libc_init_array   @ C runtime constructors
bl   main                @ your program finally starts
bx   lr                  @ only reached if main() returns
```

The copy loop is the bridge promised in Section 2: the flash copy of `.data` (the LMA, at `_sidata`) is copied to its RAM home (the VMA). The zero loop keeps `.bss`'s promise. Then — and only then — is C's world ready, and `main` is called like any other function.

> ⚠ Common misconception
>
> "`main` is where my program starts." `main` is the *third* C function your board runs (after `SystemInit` and the constructors), preceded by a stack setup and two memory loops. When a program crashes before reaching a breakpoint on `main`, this pre-main world is where it died — and now you can step through it in the Disassembly view like any other code.

> ℹ What if main returns?
>
> On a PC, returning from `main` hands control back to the OS. Here there is nobody to return *to*: the generated code just executes `bx lr` into effectively undefined behavior (and unexpected interrupts land in `Default_Handler`, an infinite loop). This is the deep reason every embedded `main` ends in `while (1)` — the program must never fall off the end of the world.

6 Into flash and out of reset: programming, boot, bootloaders

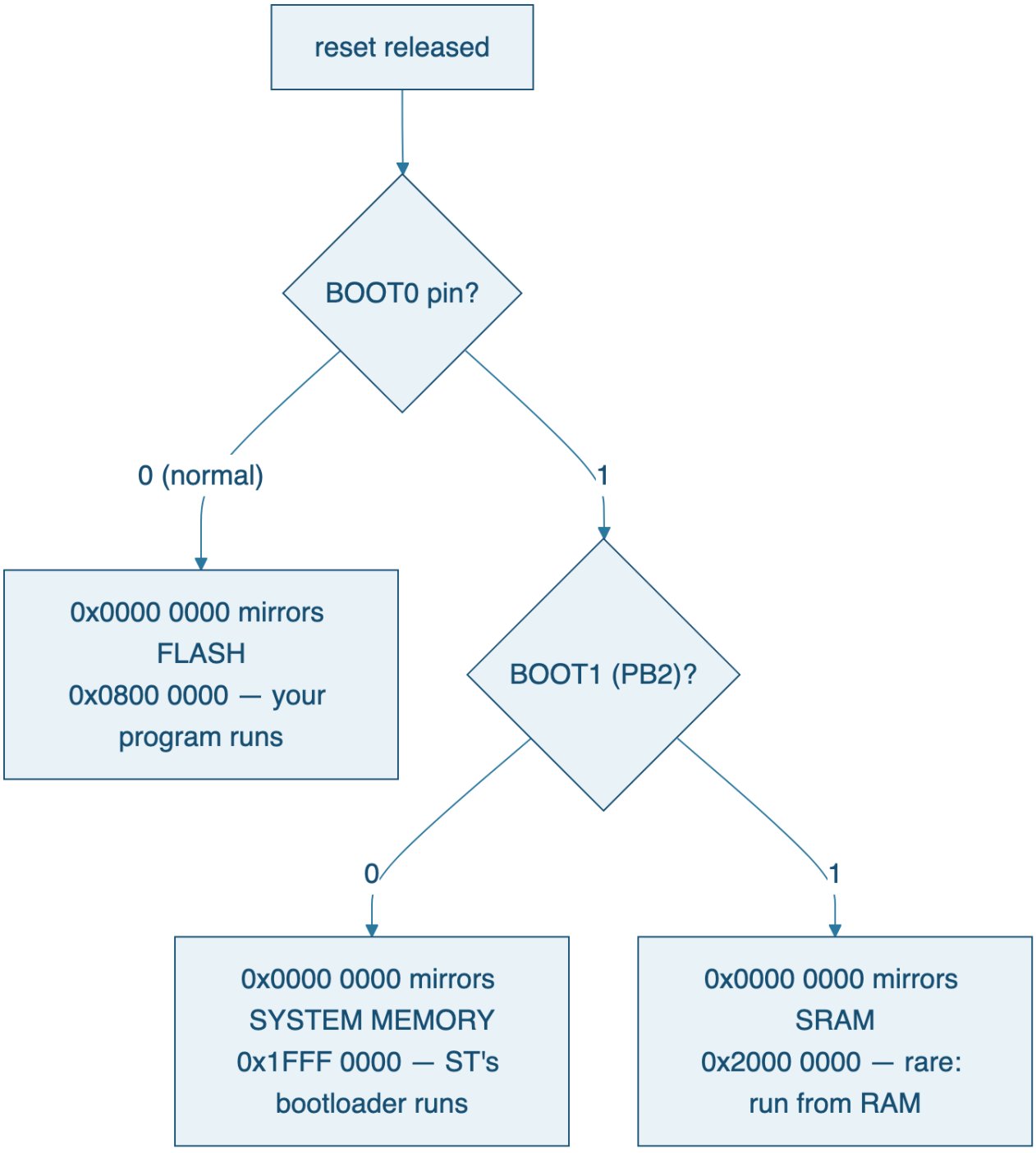
6.1 How the bytes get into flash

Flash cannot be written like RAM: it erases only in whole **sectors** (the F401RE's 512 KB is divided into sectors of 16, 64 and 128 KB) and each erase/program cycle takes real time. Something must drive that procedure — and that something is a *debug probe*: the **ST-LINK** built into the lower half of your Nucleo board. Over a two-wire protocol called **SWD**, the ST-LINK can halt the CPU, read and write any address, and run the flash controller's erase/program sequence. On the host side, the driver is STM32CubeProgrammer or the ST-LINK GDB server (both in CubeCLT); in daily use you never see them directly — pressing **F5** in VS Code does it all (Section 7).

> **i** Why your program survives unplugging
>
> After programming, the ST-LINK's job is done. The bytes sit in flash at `0x0800 0000` exactly as the linker placed them (Section 3), through power-off, forever — until the next erase. The board on your desk needs the USB cable for *power*, not for its memory.

6.2 What the chip does at power-up

Reset always begins the same way (Section 5): fetch `SP` and `PC` from the vector table at address `0x0000 0000`. But *what is at address zero*? Nothing, physically — address zero is an **alias**, a mirror whose target is chosen by two pins sampled at reset:



On the Nucleo, BOOT0 is tied low, so every reset lands in your flash image. The details are one page of the reference manual: RM0368 Rev 6, §2.4 *Boot configuration* — p. 41.

6.3 The bootloader you already own

That middle branch is worth pausing on. The **system memory** region (Section 2) contains a small program ST burned into ROM at the factory: a bootloader that can receive a new firmware image

over UART or USB DFU and write it to flash — no ST-LINK involved. Pull BOOT0 high, reset, and the chip wakes up as a self-reprogramming device (AN2606 documents the exact interfaces and pins per chip). This is how devices in the field get updated without a debug probe, and it costs you nothing: it is in ROM, it cannot be erased, and it runs only when asked.

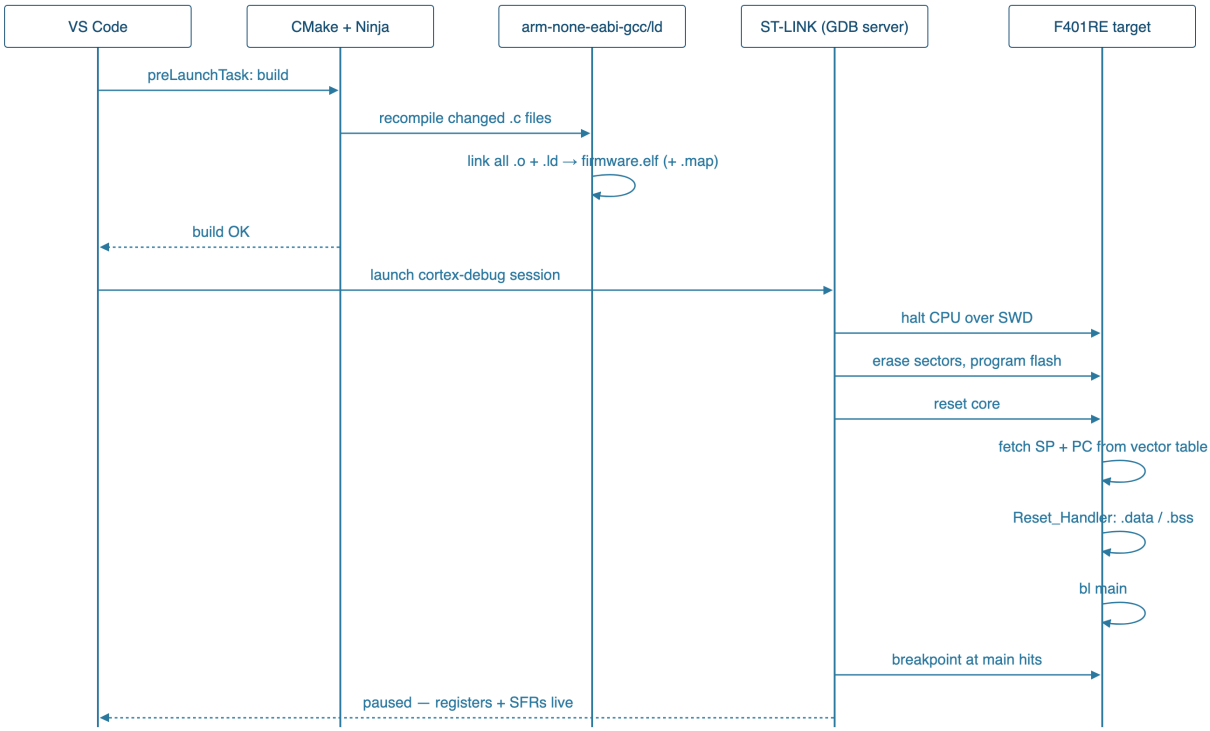
A **custom bootloader** applies the same idea with your own rules: a small program of yours placed at `0x0800 0000` that decides — based on a button, a flag, a checksum — whether to jump into the *real* application placed higher in flash (say `0x0800 8000`). The mechanics are exactly the boot protocol run by hand: read the application's vector table, load its word 0 into SP, jump to its word 1, after telling the Cortex-M where the new vector table lives (the `SCB->VTOR` register — which is also how HAL's `SystemInit` points VTOR at `0x0800 0000` on every normal boot).

> ⚠ Common misconception
>
> "The debugger downloads my program into the chip's RAM to run it." No — the ST-LINK writes it into **flash**, and the chip runs it from flash. RAM gets only `.data`'s copied values and everything writable, placed there by your own startup code at every reset (Section 5). If firmware ran from RAM, it would vanish at every power cycle.

7 What happens after I press Build

7.1 The keypress, traced end to end

Everything in this companion composes into one story. Here is **F5** in VS Code, from keypress to your breakpoint on `main`:



Step by step, with each chapter's role:

1. **Build** (Section 1, Section 4). Ninja consults its dependency graph: only the `.c` files you touched are recompiled into fresh `.o` files; the linker then always reruns, because any object may move every symbol.
2. **Place** (Section 3). The linker script assigns every byte its flash or RAM address; the `.map` file records the verdict; `firmware.elf` holds the bytes plus the debug story.
3. **Flash** (Section 6). The GDB server halts the CPU over SWD, erases exactly the sectors your image touches, programs the linker's bytes at `0x0800 0000`, and verifies them.
4. **Reset and boot** (Section 6, Section 5). BOOT0 is low, so address zero mirrors flash. The core fetches `_estack` into SP, `Reset_Handler` into PC; the startup code keeps C's promises — clock setup, `.data` copied from `_sidata`, `.bss` zeroed — and calls `main`.
5. **Debug.** The pre-set breakpoint at `main` fires. The Watch panel, the PERIPHERALS (SFR) view, and the Disassembly view you have used since L04 are all reading the target through the same SWD link that did the flashing.

Total elapsed time: about two seconds. Number of machines involved: seven. Amount of magic: zero.

> **!** Why this matters
>
> When the pipeline breaks, this timeline is your diagnostic map. *Build fails* → stages 1–2: read the first compiler or linker error, and the `.map` if a region overflowed. *Flash fails* → stage 3: the probe or a locked/busy target, not your C. *Flashes but never reaches `main`* → stage 4: something pre-main, steppable in the Disassembly view from `Reset_Handler` itself. *Runs but misbehaves* → at last, and only now, an actual bug in your program.

7.2 Where to go deeper

- The lecture references page collects every page-anchored primary source: academic references.
- RM0368 for this chip's memory, flash and boot machinery (memory map p. 38, boot p. 41, flash p. 45).
- PM0214 for the Cortex-M4 core: registers, exceptions, and every instruction the startup code uses.
- ST AN2606 (*STM32 microcontroller system memory boot mode*) for the ROM bootloader's interfaces, chip by chip.
- Your own build directory: `firmware.map`, `arm-none-eabi-objdump -d`, and `arm-none-eabi-nm` — the primary sources for *your* program.